DevOps Project Documentation(Two-Tier Flask Application Deployment on AWS)

1. Introduction

1.1 Purpose

The purpose of this technical document is to describe the design, deployment, and operational processes involved in implementing a **two-tier web application architecture on AWS using DevOps practices**.

This document explains the infrastructure setup, CI/CD pipeline, monitoring, security implementation, and operational practices followed during the deployment of the application.

The intended audience includes:

- Chief Technology Officer (CTO)
- DevOps Engineers
- Software Developers
- Solution Architects
- Cloud Engineers
- Resellers and technical stakeholders

This documentation serves as a reference for understanding the system architecture, deployment workflow, and DevOps practices used to ensure scalability, security, and reliability.

1.2 Scope

This document covers the **end-to-end DevOps implementation** for deploying a two-tier application on AWS.

The scope includes:

- Infrastructure setup using **AWS EC2**
- Containerization using **Docker**
- CI/CD automation using **GitHub Actions**
- Image storage using **Amazon ECR**
- Monitoring using **Prometheus and Grafana**
- Security implementation using **Nginx and SSL certificates**
- Best practices for system reliability and monitoring

This document focuses on the **deployment and operational lifecycle of the application**.

---

2. Overview of DevOps Project

2.1 Definition

DevOps is a set of practices that combines software development and IT operations to shorten the development lifecycle while delivering high-quality software continuously.

The DevOps approach enables:

- Faster deployments
- Automated workflows
- Continuous integration and delivery
- Improved system reliability
- Enhanced collaboration between development and operations teams

In this project, DevOps principles were applied to automate the **build, deployment, monitoring, and maintenance of a two-tier web application**.

2.2 Key Principles

The deployment follows the **AWS Well-Architected Framework**, which consists of six pillars.

1. Operational Excellence

Operational excellence focuses on running and monitoring systems effectively.

Implementation:

- CI/CD pipeline using **GitHub Actions**
- Automated Docker builds
- Infrastructure monitoring using **Prometheus and Grafana**

---

2. Security

Security ensures protection of applications and infrastructure.

Implementation:

- IAM user with limited permissions
- Secure access using **SSH**
- HTTPS encryption using **Nginx and SSL**
- AWS Security Groups for network control

---

3. Reliability

Reliability ensures the system continues operating even during failures.

Implementation:

- Containerized services using Docker
- Monitoring dashboards
- System health metrics using Node Exporter

---

4. Performance Efficiency

Performance efficiency ensures optimal resource usage.

Implementation:

- Lightweight containers
- Efficient monitoring tools
- Resource metrics collection

---

5. Cost Optimization

Cost optimization focuses on minimizing infrastructure costs.

Implementation:

- Use of **t2.micro EC2 instance**
- Open-source monitoring tools
- Containerized deployment to reduce infrastructure usage

---

6. Sustainability

Sustainability focuses on efficient resource usage.

Implementation:

- Docker containers reduce resource consumption

- Optimized deployment process

2.3 Goals and Objectives

The main objectives of this DevOps project are:

- Deploy a **two-tier web application on AWS**
- Containerize the application using Docker
- Implement CI/CD automation
- Store Docker images in Amazon ECR
- Monitor system metrics using Prometheus and Grafana
- Secure the application using HTTPS
- Follow DevOps best practices

2.4 Constraints and Assumptions

Constraints

- Limited budget for infrastructure
- Time constraint for project implementation
- Limited compute resources

Assumptions

- AWS account is available
- GitHub repository is accessible
- Docker environment is supported on EC2

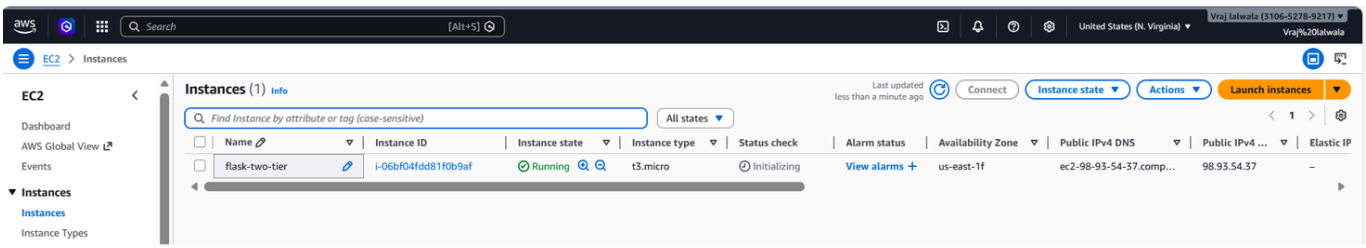
---

3. Architecture and Infrastructure

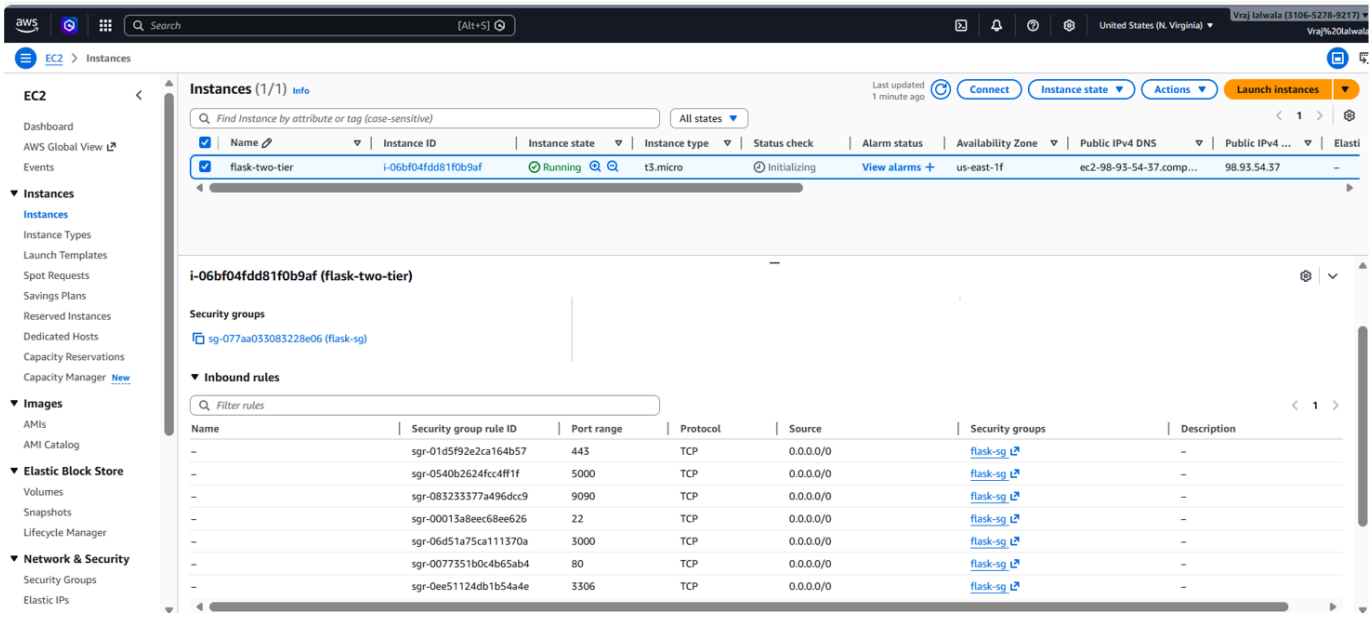
3.1 Infrastructure Overview

The infrastructure consists of the following components:

- AWS EC2 instance hosting the application
- Docker containers for application services
- Amazon ECR for storing container images
- GitHub Actions for CI/CD automation
- Prometheus and Grafana for monitoring
- Nginx for secure reverse proxy



Security Group Rules:-



3.2 Environments

The DevOps workflow includes the following environments:

Development Environment

Used for writing and testing application code.

Production Environment

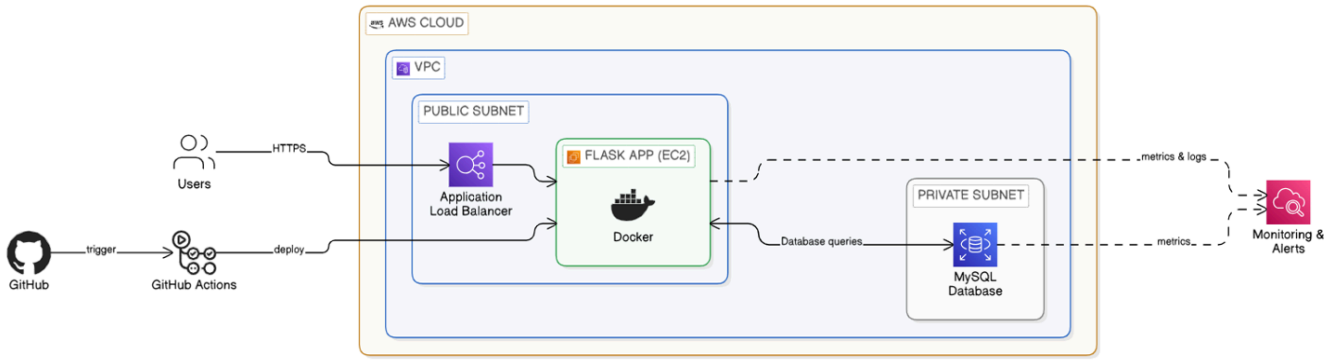
Used for deploying the application on AWS EC2 for live usage.

3.4 Application Architecture & User Request Flow

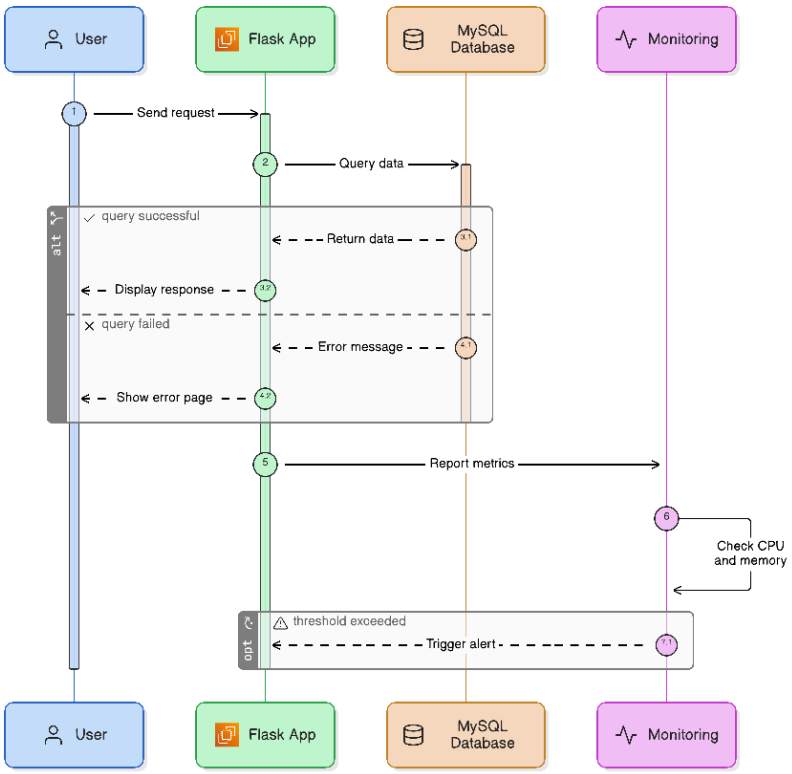
Application architecture:

```
User
  |
Internet
  |
AWS EC2 Instance
  |
Nginx Reverse Proxy (HTTPS)
  |
Docker Containers
 ├── Flask Application
 ├── MySQL Database
 ├── Prometheus
 └── Grafana
  |
Amazon ECR
  |
GitHub Actions CI/CD
  |
GitHub Repository
```

Architecture Diagram:-



User Request Flow Diagram:-



3.3 Tools and Technologies

Tool	Purpose
AWS EC2	Cloud infrastructure
Docker	Containerization
Amazon ECR	Docker image registry
GitHub	Version control
GitHub Actions	CI/CD automation
Flask	Web application
MySQL	Database

Prometheus	Metrics collection
Grafana	Monitoring dashboard
Node Exporter	System metrics
Nginx	Reverse proxy
Certbot	SSL certificate generation

---

4. Development Practices

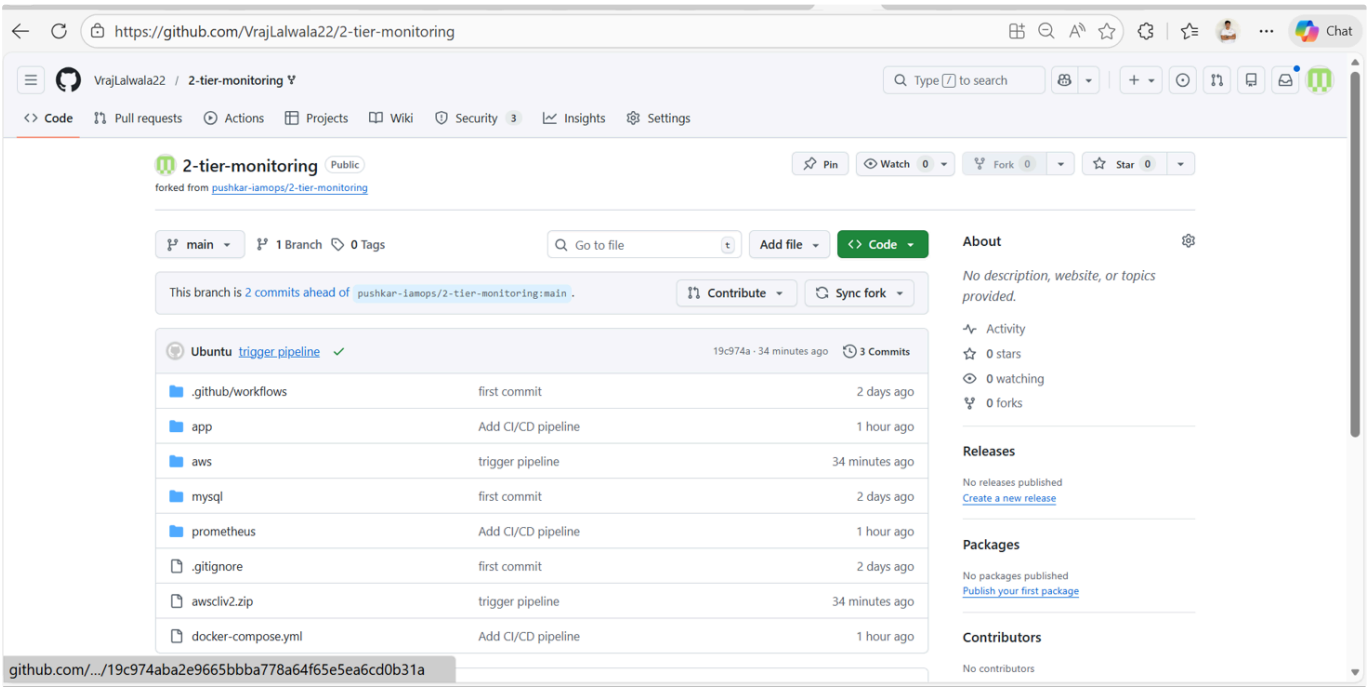
4.1 Version Control

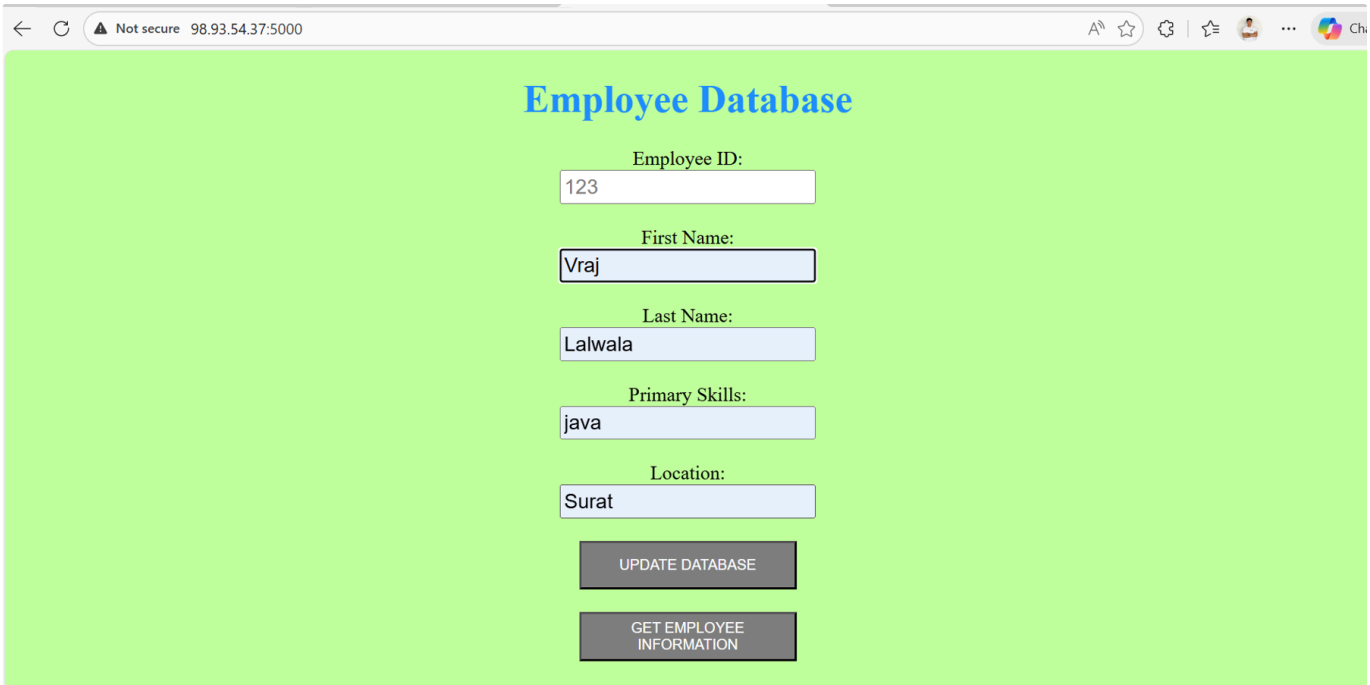
Version control is managed using **Git and GitHub**.

Practices followed:

- Source code stored in GitHub repository
- Code changes committed using Git
- Branching strategy using **main branch**
- Collaboration through version control

GitHub Repository:-





Confirmation on mysql dataset:-

```
mysql> select * from employee;
+--------+------------+-----------+---------------+----------+
| emp_id | first_name | last_name | primary_skill | location |
+--------+------------+-----------+---------------+----------+
| 1      | Amanda     | Williams  | Smile         | local    |
| 1      | Alan       | Williams  | Empathy       | alien    |
| 1      | Amanda     | Williams  | Smile         | local    |
| 1      | Alan       | Williams  | Empathy       | alien    |
| 1      | Amanda     | Williams  | Smile         | local    |
| 1      | Alan       | Williams  | Empathy       | alien    |
| 123    | Vraj       | Lalwala   | java          | Surat    |
+--------+------------+-----------+---------------+----------+
7 rows in set (0.00 sec)

mysql>
```

4.2 Continuous Integration

Continuous Integration was implemented using **GitHub Actions**.

Workflow:

1. Code pushed to GitHub
2. GitHub Actions pipeline triggers
3. Docker images built
4. Images pushed to Amazon ECR

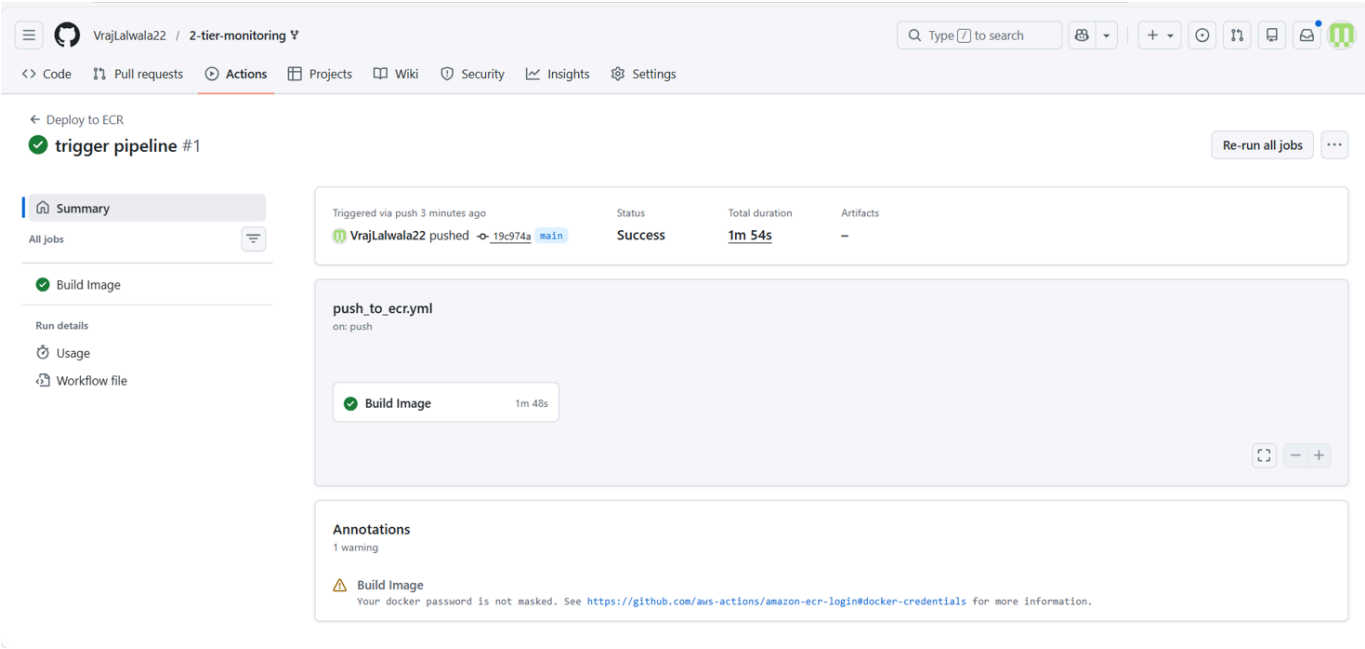
SSH-Key Generation for Automation:-

```
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ubuntu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ubuntu/.ssh/id_ed25519
Your public key has been saved in /home/ubuntu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:SQBkRGNUoKyRlV0unnAEyhrtkfP75IViX/GVk104l/0 ubuntu@ip-172-31-70-193
The key's randomart image is:
+--[ED25519 256]--+
|  oO%=+          |
|.*.*.o .       .o|
|=.O o . .     o.+|
|.= B o . .   + +.|
|o . +   S   = . E|
|     . . o . .   |
|    + o o .      |
|   . * o         |
|      +          |
+----[SHA256]-----+
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ cat ~/.ssh/id_rsa.pub
cat: /home/ubuntu/.ssh/id_rsa.pub: No such file or directory
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJ3DiSsSXVNHxjLodnMO6jTL57RcNbr+xWXj8BtoRgA5 ubuntu@ip-172-31-70-193
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ nano ~/.ssh/authorized_keys
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$
```

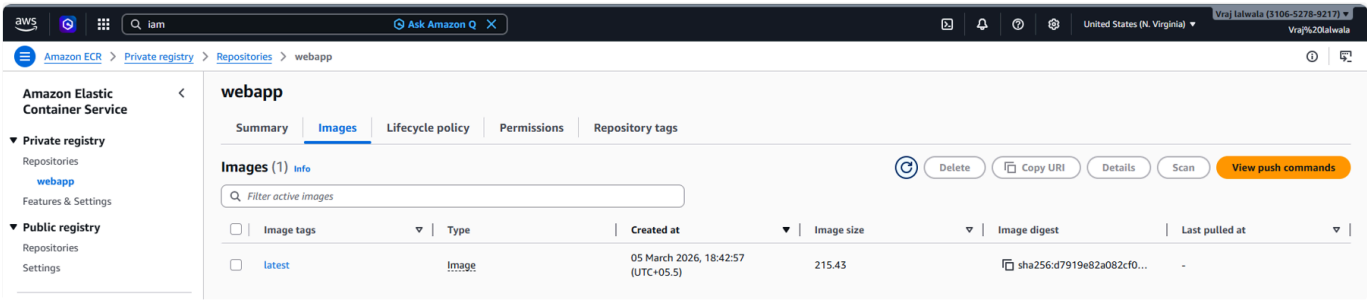
Commiting message on our Github Repo:-

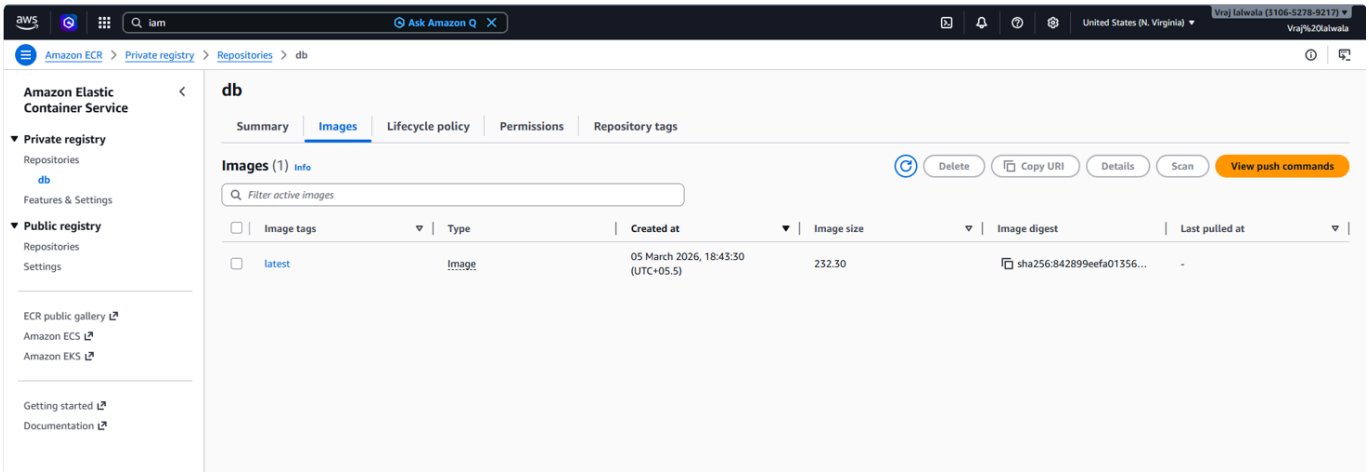
```
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ git add .
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ git commit -m "trigger pipeline"
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ git push origin main
Username for 'https://github.com': VrajLalwala22
Password for 'https://VrajLalwala22@github.com':
Enumerating objects: 8446, done.
Counting objects: 100% (8446/8446), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5332/5332), done.
Writing objects: 100% (8442/8442), 122.84 MiB | 40.60 MiB/s, done.
Total 8442 (delta 2659), reused 8441 (delta 2658), pack-reused 0
remote: Resolving deltas: 100% (2659/2659), completed with 1 local object.
remote: warning: See https://gh.io/lfs for more information.
remote: warning: File awscliv2.zip is 63.75 MB; this is larger than GitHub's recommended maximum file size of 50.00 MB
remote: warning: GH001: Large files detected. You may want to try Git Large File Storage - https://git-lfs.github.com.
To https://github.com/VrajLalwala22/2-tier-monitoring.git
   95f9cde..19c974a  main -> main
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$
```

GitHub Actions Workflow:-



Confirmation with ECR Repo:-





4.3 Automated Testing

Basic application validation tests were performed by:

- Running containers
- Verifying database connectivity
- Testing application endpoints

Future improvements may include automated test pipelines.

4.4 Code Quality and Analysis

Code quality practices include:

- Organized repository structure
- Modular application code
- Version control management

---

5. Deployment and Release Management

5.1 Deployment Strategy

The deployment uses **container-based deployment**.

Strategy:

- Docker images built via CI pipeline
- Images stored in Amazon ECR
- EC2 pulls the latest images
- Containers deployed using Docker

5.2 Configuration Management

Configuration parameters are passed through environment variables.

Examples:

```
DBHOST
DBUSER
DBPORT
DATABASE
```

Docker networking ensures service communication.

5.3 Continuous Delivery/Deployment

Continuous delivery is implemented using **GitHub Actions**.

Process:

```
Code Push → GitHub Actions → Build Docker Images → Push to ECR → Deploy on EC2
```

---

6. Monitoring and Incident Management

6.1 Monitoring Tools and Metrics

Monitoring was implemented using:

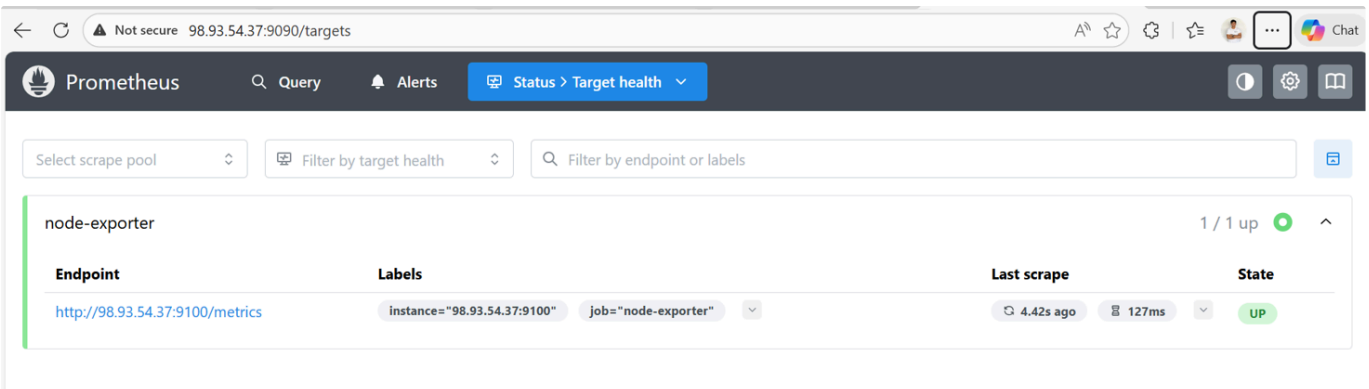
- Prometheus
- Grafana
- Node Exporter

Metrics collected include:

- CPU usage
- Memory utilization
- System load

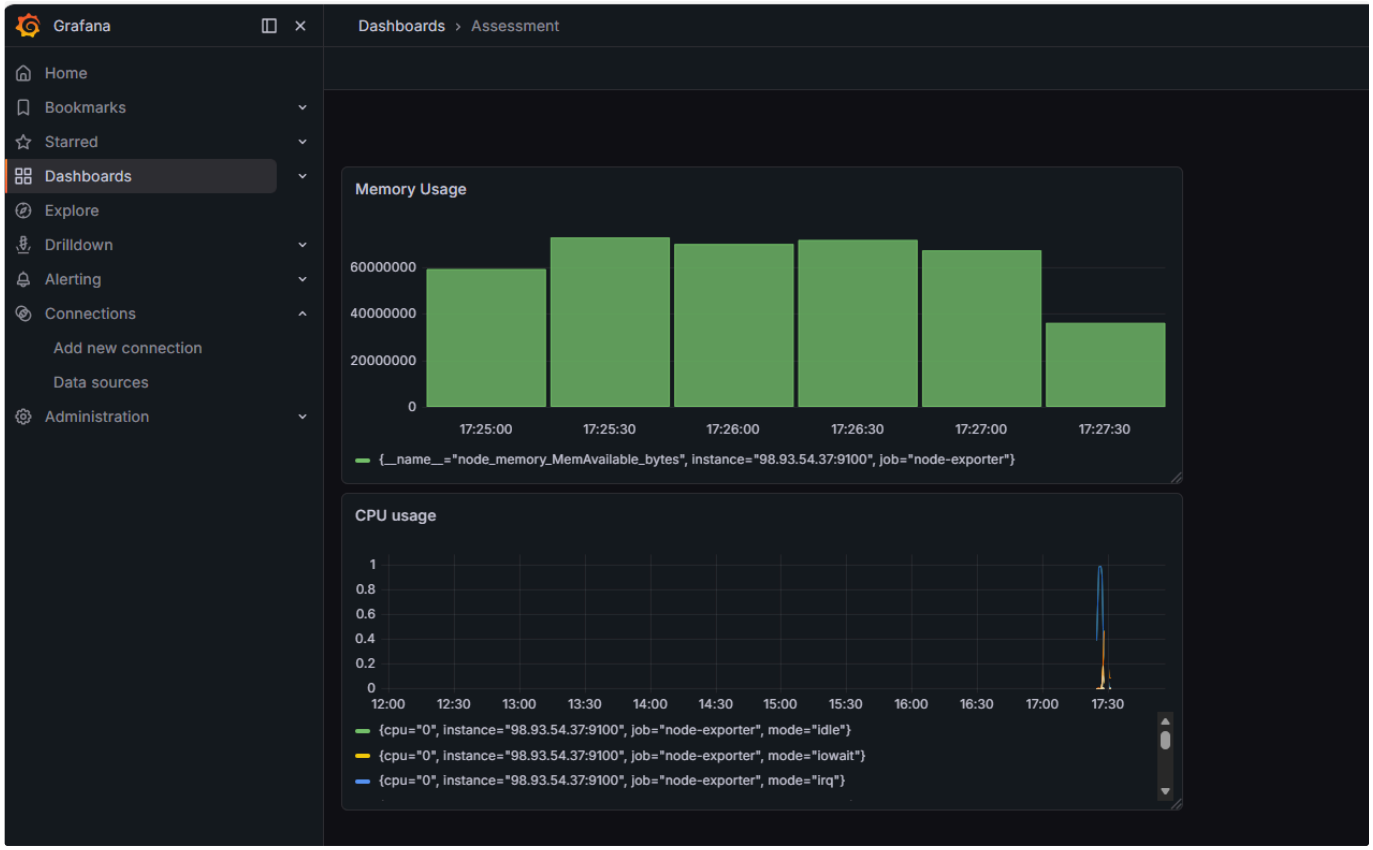
Prometheus Targets:-

```
ubuntu@ip-172-31-70-193:~/2-tier-monitoring/prometheus$ docker run -d \
--name prometheus \
-p 9090:9090 \
-v $(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml \
prom/prometheus
c2b8fbf0b70d61b2271a328017e74d0f0fb0d6c20086ba30694d8042b383f57b
ubuntu@ip-172-31-70-193:~/2-tier-monitoring/prometheus$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
                  NAMES
c2b8fbf0b70d   prom/prometheus      "/bin/prometheus --c…"   4 seconds ago    Up 4 seconds    0.0.0.0:9090->9090/tcp, [::]:9090->9090
/tcp              prometheus
9d43d87f919d   prom/node-exporter   "/bin/node_exporter"     15 minutes ago   Up 15 minutes   0.0.0.0:9100->9100/tcp, [::]:9100->9100
/tcp              node-exporter
1eefae8f7fcc   flask-employee       "python3 app.py"         24 minutes ago   Up 24 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000
/tcp, 8080/tcp    flask-app
aad60ded3af0   mysql-employee       "docker-entrypoint.s…"   30 minutes ago   Up 30 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306
/tcp, 33060/tcp   mysql-db
ubuntu@ip-172-31-70-193:~/2-tier-monitoring/prometheus$
```



Grafana Dashboard:-

```
ubuntu@ip-172-31-70-193:~/2-tier-monitoring/prometheus$ docker run -d \
--name grafana \
-p 3000:3000 \
grafana/grafana
Unable to find image 'grafana/grafana:latest' locally
latest: Pulling from grafana/grafana
589002ba0eae: Pull complete
dce812085769: Pull complete
28c53581fa0d: Pull complete
8cec4fd2d8f1: Pull complete
2898b87e636f: Pull complete
103f1b507f08: Pull complete
42661385b0ec: Pull complete
ac89855d60f9: Pull complete
2ce346fa65f7: Pull complete
a1cfed684a05: Pull complete
4f4fb700ef54: Pull complete
ba30534b47b1: Pull complete
Digest: sha256:b0ae311af06228bcfd4a620504b653db80f5b91e94dc3dc2a5b7dab202bcde20
Status: Downloaded newer image for grafana/grafana:latest
f18768ac9be2d0dc0591a1ee61c5f9a4a42529bbdb9654f9eaeaa28690bd308b
ubuntu@ip-172-31-70-193:~/2-tier-monitoring/prometheus$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
       NAMES
f18768ac9be2   grafana/grafana      "/run.sh"                6 seconds ago    Up 6 seconds    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
       grafana
c2b8fbf0b70d   prom/prometheus      "/bin/prometheus --c…"   2 minutes ago    Up 2 minutes    0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
       prometheus
9d43d87f919d   prom/node-exporter   "/bin/node_exporter"     18 minutes ago   Up 18 minutes   0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp
       node-exporter
1eefae8f7fcc   flask-employee       "python3 app.py"         26 minutes ago   Up 26 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp, 8080/
tcp    flask-app
aad60ded3af0   mysql-employee       "docker-entrypoint.s…"   33 minutes ago   Up 33 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060
/tcp   mysql-db
ubuntu@ip-172-31-70-193:~/2-tier-monitoring/prometheus$
```



6.2 Alerting and Notification

Alerts can be configured in Grafana to notify administrators when:

- CPU usage exceeds threshold
- Memory usage is high
- System performance degrades

6.3 Incident Management

Incident handling process:

1. Monitoring detects abnormal metrics
2. Alerts are triggered
3. DevOps team investigates logs
4. Containers restarted or redeployed if necessary

---

7. Security & Compliance

Security measures implemented:

- IAM user with least privilege access
- Secure SSH access to EC2
- Configured NGINX reverse proxy
- HTTPS encryption using SSL certificates
- Security groups controlling network traffic

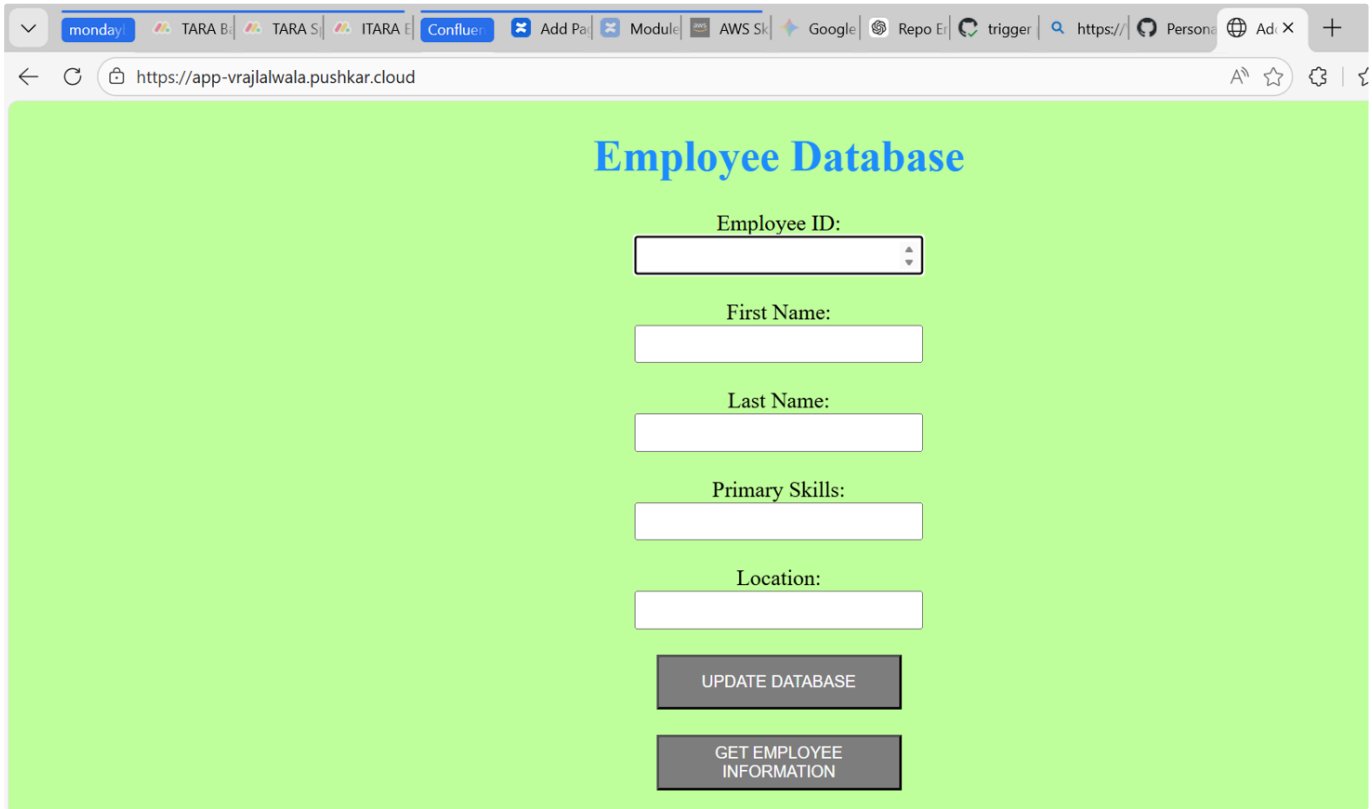
NGINX Reverse Proxy:-

```
ubuntu@ip-172-31-70-193: ~/

  GNU nano 7.2                                    /etc/nginx/sites-available/flaskapp
server {
    listen 80;
    server_name app-vrajlalwala.pushkar.cloud;

    location / {
        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

CertBot Certification:-

```
ubuntu@ip-172-31-70-193: ~/

ubuntu@ip-172-31-70-193:~/2-tier-monitoring$ sudo certbot --nginx
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address (used for urgent renewal and security notices)
 (Enter 'c' to cancel): vraj.lalwala@itara.io

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Please read the Terms of Service at
https://letsencrypt.org/documents/LE-SA-v1.6-August-18-2025.pdf. You must agree
in order to register with the ACME server. Do you agree?
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(Y)es/(N)o: y

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Would you be willing, once your first certificate is successfully issued, to
share your email address with the Electronic Frontier Foundation, a founding
partner of the Let's Encrypt project and the non-profit organization that
develops Certbot? We'd like to send you email about our work encrypting the web,
EFF news, campaigns, and ways to support digital freedom.
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(Y)es/(N)o: y
Account registered.

Which names would you like to activate HTTPS for?
We recommend selecting either all domains, or all domains in a VirtualHost/server block.
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
1: app-vrajlalwala.pushkar.cloud
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Select the appropriate numbers separated by commas and/or spaces, or leave input
blank to select all options shown (Enter 'c' to cancel):
Requesting a certificate for app-vrajlalwala.pushkar.cloud

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/app-vrajlalwala.pushkar.cloud/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/app-vrajlalwala.pushkar.cloud/privkey.pem
This certificate expires on 2026-06-03.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for app-vrajlalwala.pushkar.cloud to /etc/nginx/sites-enabled/flaskapp
Congratulations! You have successfully enabled HTTPS on https://app-vrajlalwala.pushkar.cloud

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
If you like Certbot, please consider supporting our work by:
 * Donating to ISRG / Let's Encrypt:   https://letsencrypt.org/donate
 * Donating to EFF:                    https://eff.org/donate-le
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
ubuntu@ip-172-31-70-193:~/2-tier-monitoring$
```

HTTPS Enabled Application:-



---

8. Cost Optimisation

Cost optimization techniques used:

- Using **t2.micro EC2 instance**
- Using open-source monitoring tools
- Using containerized infrastructure

Estimated Cost Inputs

- EC2 compute usage
- ECR storage
- Data transfer

Business Value

- Reduced infrastructure cost
- Automated deployment
- Improved operational efficiency

---

10. Implementation Timeline

10.1 Milestones and Deliverables

Phase 1: Infrastructure setup – 1 hour
Phase 2: Docker container deployment – 1 hour
Phase 3: CI/CD pipeline implementation – 1 hour
Phase 4: Monitoring and SSL configuration – 1 hour

Final Delivery: Completed within project timeframe.

---

11. Continuous Improvement

11.1 Feedback and Retrospectives

Feedback was gathered from system performance monitoring and deployment results to identify improvement opportunities.

11.2 Iterative Enhancements

Future improvements may include:

- Kubernetes deployment
- Infrastructure as Code using Terraform
- Automated scaling
- Centralized logging systems

---

12. Conclusion

12.1 Summary

This project successfully demonstrates a **DevOps-based deployment of a two-tier web application on AWS**.

Key achievements include:

- Containerized application deployment
- CI/CD automation using GitHub Actions
- Image management using Amazon ECR
- Monitoring using Prometheus and Grafana
- Secure HTTPS implementation

---

13. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____